

Testing Guide — EMS Readiness Optimization

Quick Start

```
# Run all tests
python -m pytest tests/ -v

# Run a specific test module
python -m pytest tests/test_demand_advanced.py -v

# Run tests matching a keyword
python -m pytest tests/ -k "distance" -v

# Run with coverage report
python -m pytest tests/ --cov=src/ems_readiness --cov-report=term-missing
```

Test Organisation

Tests are organised by **concern**, not by source file.

File	Category	Count	Description
test_simulation_core.py	Unit	12	Sim engine init, arrivals, event ordering, unit conservation
test_dispatch_logic.py	Unit	7	Dispatcher nearest-available logic
test_extreme_cases.py	Unit	5	Boundary conditions (0 units, huge demand)
test_reproducibility.py	Unit	4	Seed-based determinism of the simulation
test_demand_advanced.py	Unit	14	NHPP generation, lambda table loading, effective-rate calculation
test_service_advanced.py	Unit	18	Distance matrices, travel time, service-time distributions
test_optimization_advanced.py	Unit	16	PuLP model build, MCLP/p-median, policy helpers
test_simulation_advanced.py	Unit	16	MetricsCollector, UnitPool, EMSUnit, IncidentEntity
test_seed_manager.py	Unit	15	SeedManager determinism, isolation, config loading
test_integration.py	Integration	6	End-to-end pipeline, batch runner, config-driven runs
test_properties.py	Property	10	Hypothesis-driven invariants (distances, rates, metrics)
test_regression.py	Regression	9	Golden-value locks for known-good outputs

File	Category	Count	Description
<code>test_performance.py</code>	Performance	6	Wall-clock benchmarks with configurable thresholds

Total: 138+ tests across 13 files.

Test Categories

Unit Tests

Focused on a single function or class. Mocking is used sparingly — only for external I/O such as file reads or heavy SimPy processes.

Integration Tests

Exercise a full or near-full pipeline: config → demand generation → simulation → metrics collection. These verify that modules compose correctly.

Property-Based Tests

Use the [Hypothesis](https://hypothesis.readthedocs.io/) (<https://hypothesis.readthedocs.io/>) library to generate random inputs and verify **invariants** that must hold for all valid inputs:

- Distance functions return non-negative values
- Effective demand rate scales linearly with factors
- Metrics remain within physical bounds

Regression Tests

Lock specific numerical outputs (“golden values”) so that future refactors don’t silently change results. If a change is intentional, update the expected value and document **why** in the commit message.

Performance Tests

Assert that key operations stay within wall-clock budgets:

```
| Operation | Budget |
|-----|-----|
| Single simulation run | < 5 s |
| Batch of 5 replications | < 30 s |
| Optimisation model build | < 2 s |
| Distance matrix (50 × 50) | < 1 s |
| NHPP arrival generation (7 days) | < 1 s |
| Lambda table loading | < 1 s |
```

Adjust thresholds in `test_performance.py` if the CI environment is slower.

Pipeline Enhancement Features

The data pipeline includes several quality-of-life enhancements that can be tested independently:

Data Validation

```
# Test validation directly
python -c "
from scripts.data_processing.validation import validate_raw_data
ok, errors = validate_raw_data('.')
print('OK:', ok, '| Errors:', len(errors))
"
```

Smart Caching

```
# Check cache status
python -c "
from scripts.data_processing.cache import CacheManager
cm = CacheManager('.')
print(cm.summary())
"

# Second run should skip unchanged tiers
python scripts/generate_all_data.py # First run
python scripts/generate_all_data.py # Second run (cached, much faster)
```

Data Versioning

```
# View current data version
python scripts/generate_all_data.py --version

# Compare manifests programmatically
python -c "
from scripts.data_processing.versioning import DataVersionManager
dvm = DataVersionManager('.')
result = dvm.compare_manifests()
print('Needs regen:', result['needs_regeneration'])
for r in result['reasons']:
    print(' -', r)
"
```

Parallel Processing

```
# Run with parallel workers
python scripts/generate_all_data.py --force --jobs 4
```

Adding New Tests

1. **Pick the right file** based on the category table above.
2. **Follow the naming convention:** `test_<module>_<concern>.py` or add to an existing file.
3. **Use fixtures from `conftest.py`** — `simulation_config`, `base_allocation`, `simulation_engine` are pre-built.
4. **For property tests**, import from `hypothesis` and decorate with `@given(...)`. Keep `max_examples` reasonable (≤ 50) for CI speed.
5. **For regression tests**, document the expected value's source in a comment.

Continuous Integration Tips

```
# Fast smoke test (exclude slow perf tests)
python -m pytest tests/ -v -k "not performance" --timeout=30

# Full suite including performance
python -m pytest tests/ -v --timeout=60
```

Coverage Goals

Module	Target	Notes
demand/	$\geq 85\%$	Core NHPP + lambda logic
service/	$\geq 80\%$	Distance, travel, service-time
optimization/	$\geq 75\%$	Model build paths; solver internals hard to cover
simulation/	$\geq 80\%$	Engine, entities, metrics
utils/	$\geq 90\%$	Reproducibility, helpers